Отчёт по лабораторной работе № 1

Цветовые модели: CMYK – RGB – HLS

Студент: [ФИО]

Группа: [номер группы]

Вариант: нечётный (CMYK – RGB – HLS)

Репозиторий: [ссылка на GitHub, например https://github.com/username/color-models]

1. Цель и задание

Изучить цветовые модели RGB, CMYK и HLS и преобразования между ними. Создать веб-приложение, позволяющее выбирать цвет и интерактивно менять его, показывая составляющие сразу в трёх моделях.

Требования: ввод точных значений в поля, выбор цвета из палитры, плавное изменение компонент ползунками; при изменении любой компоненты две другие модели пересчитываются автоматически.

2. Теоретические сведения

2.1. RGB

Аддитивная модель: цвет получается смешением красного, зелёного и синего света. Каждая компонента принимает значения от 0 до 255. Модель используется в мониторах; в программе она служит центральным представлением, через которое выполняются все остальные переводы.

2.2. CMYK

Субтрактивная модель: голубой (C), пурпурный (M), жёлтый (Y) и чёрный (K) красители поглощают свет. Компоненты задаются в процентах (0–100). Чёрный компонент вводится отдельно, так как смесь трёх красителей не даёт чистого чёрного, а чёрная краска дешевле.

RGB → CMYK (R, G, B нормированы в диапазон [0, 1]):

$$K = 1 - max(R,\ G,\ B), \quad C = (1 - R - K) \,/\, (1 - K), \quad M = (1 - G - K) \,/\, (1 - K), \quad Y = (1 - B - K) \,/\, (1 - K)$$

Если K = 1 (чёрный цвет), то C = M = Y = 0. Обратный перевод:

$$R = 255 \cdot (1 - C) \cdot (1 - K), \quad G = 255 \cdot (1 - M) \cdot (1 - K), \quad B = 255 \cdot (1 - Y) \cdot (1 - K)$$

2.3. HLS

Модель, близкая к восприятию человека: H (Hue, тон) — угол на цветовом круге, 0–360°; L (Lightness, светлота) — от 0 (чёрный) до 100% (белый); S (Saturation, насыщенность) — от 0 (серый) до 100%. Чистый цвет лежит на L = 50%.

RGB → HLS (Δ = max − min):

$$L = (max + min) \,/\, 2, \quad S = \Delta \,/\, (1 - |2L - 1|), \quad S = 0 \text{ при } \Delta = 0$$

$$H = 60°\cdot(((G - B)/\Delta) \bmod 6), \text{ если } max = R;\quad 60°\cdot((B - R)/\Delta + 2), \text{ если } max = G;\quad 60°\cdot((R - G)/\Delta + 4), \text{ если } max = B$$

Если H < 0, прибавляется 360°; при Δ = 0 принимается H = 0 (тон серого не определён). Обратный перевод:

$$a = S\cdot min(L,\ 1 - L),\quad k(n) = (n + H/30) \bmod 12,\quad f(n) = L - a\cdot max(-1,\ min(k - 3,\ 9 - k,\ 1))$$

$$R = 255\cdot f(0),\quad G = 255\cdot f(8),\quad B = 255\cdot f(4)$$

3. Реализация

Программа написана на языке Go с использованием только стандартной библиотеки. Приложение состоит из одного файла main.go.

- Функции rgbToCMYK, cmykToRGB, rgbToHLS, hlsToRGB и hue реализуют формулы из раздела 2.
- HTTP-обработчик /api/convert принимает название модели и значения, переводит их в RGB, затем из RGB вычисляет все остальные модели и возвращает результат в формате JSON.
- Страница (HTML, CSS, JavaScript, встроена в main.go) отвечает только за ввод и отображение: при любом изменении поля, ползунка или палитры отправляет запрос и обновляет остальные модели.
- Редактируемая модель не перезаписывается результатом пересчёта, чтобы значения не «прыгали» (например, H при S = 0).
- Вводимые значения ограничиваются допустимыми диапазонами.

Запуск: go run main.go, затем открыть http://localhost:8080.

4. Интерфейс и способы задания цвета

В каждой из трёх моделей цвет можно задать тремя способами: числовым полем, ползунком и выбором из палитры (элемент выбора цвета, как в графических редакторах). Образец цвета и его HEX-код отображаются в верхней части страницы.

[Здесь вставьте скриншот интерфейса приложения]

5. Тестирование

Корректность преобразований проверена на контрольных значениях:

Ввод (модель)	Ожидаемый результат	Результат программы
CMYK (0, 100, 100, 0)	RGB (255, 0, 0)	RGB (255, 0, 0); HLS (0°, 50%, 100%)
HLS (120°, 50%, 100%)	RGB (0, 255, 0)	RGB (0, 255, 0); CMYK (100, 0, 100, 0)
RGB (128, 128, 128)	S = 0, H = 0, K ≈ 50%	HLS (0°, 50.2%, 0%); CMYK (0, 0, 0, 49.8)
RGB (91, 75, 219)	HEX #5B4BDB	CMYK (58.4, 65.8, 0, 14.1); HLS (246.7°, 57.6%, 66.7%)

6. Вывод

В ходе работы изучены модели RGB, CMYK и HLS, получены и реализованы формулы перевода между ними. Создано приложение с тремя способами задания цвета в каждой модели и автоматическим пересчётом всех моделей при изменении любой компоненты.

7. Ссылка на репозиторий

[ссылка на GitHub]

Приложение. Листинг программы (main.go)

```
// Лабораторная работа 1. Цветовые модели: CMYK – RGB – HLS.
// Вся математика преобразований написана на Go (только стандартная библиотека).
// Запуск: go run main.go  →  http://localhost:8080
package main

import (
    "encoding/json"
    "fmt"
    "log"
    "math"
    "net/http"
    "strconv"
    "strings"
)

func clamp(x, lo, hi float64) float64 { return math.Max(lo, math.Min(hi, x)) }

// ---------- CMYK (значения 0..100 %) ----------

func rgbToCMYK(r, g, b float64) [4]float64 {
    r, g, b = r/255, g/255, b/255
    k := 1 - math.Max(r, math.Max(g, b))
    if k >= 1 { // чёрный цвет
        return [4]float64{0, 0, 0, 100}
    }
    return [4]float64{
        (1 - r - k) / (1 - k) * 100,
        (1 - g - k) / (1 - k) * 100,
        (1 - b - k) / (1 - k) * 100,
        k * 100,
    }
}

func cmykToRGB(c, m, y, k float64) [3]float64 {
    c, m, y, k = c/100, m/100, y/100, k/100
    return [3]float64{
        255 * (1 - c) * (1 - k),
        255 * (1 - m) * (1 - k),
        255 * (1 - y) * (1 - k),
    }
}

// ---------- HLS (H: 0..360°, L и S: 0..100 %) ----------

// hue вычисляет тон по RGB (0..255); общий для HSV и HLS.
func hue(r, g, b float64) float64 {
    mx, mn := math.Max(r, math.Max(g, b)), math.Min(r, math.Min(g, b))
    d := mx - mn
    if d == 0 {
        return 0 // серый: тон не определён
    }
    var h float64
    switch mx {
    case r:
        h = 60 * math.Mod((g-b)/d, 6)
    case g:
        h = 60 * ((b-r)/d + 2)
    default:
        h = 60 * ((r-g)/d + 4)
    }
    if h < 0 {
        h += 360
    }
    return h
```

```
}

func rgbToHLS(r, g, b float64) [3]float64 {
    mx, mn := math.Max(r, math.Max(g, b))/255, math.Min(r, math.Min(g, b))/255
    d := mx - mn
    l := (mx + mn) / 2
    s := 0.0
    if d != 0 {
        s = d / (1 - math.Abs(2*l-1))
    }
    return [3]float64{hue(r, g, b), l * 100, s * 100}
}

func hlsToRGB(h, l, s float64) [3]float64 {
    l, s = l/100, s/100
    a := s * math.Min(l, 1-l)
    f := func(n float64) float64 {
        k := math.Mod(n+h/30, 12)
        return 255 * (l - a*math.Max(-1, math.Min(math.Min(k-3, 9-k), 1)))
    }
    return [3]float64{f(0), f(8), f(4)}
}

// ---------- HTTP ----------

type response struct {
    RGB  [3]float64 `json:"RGB"`
    CMYK [4]float64 `json:"CMYK"`
    HLS  [3]float64 `json:"HLS"`
    Hex  string     `json:"hex"`
}

// convert: /api/convert?model=RGB|CMYK|HLS&v=a,b,c[,d]
// Переводит входную модель в RGB, а затем RGB — во все остальные модели.
func convert(w http.ResponseWriter, r *http.Request) {
    model := r.URL.Query().Get("model")
    var v []float64
    for _, p := range strings.Split(r.URL.Query().Get("v"), ",") {
        x, err := strconv.ParseFloat(p, 64)
        if err != nil {
            http.Error(w, "bad number", http.StatusBadRequest)
            return
        }
        v = append(v, x)
    }
    var rgb [3]float64
    switch {
    case model == "RGB" && len(v) == 3:
        copy(rgb[:], v)
    case model == "CMYK" && len(v) == 4:
        rgb = cmykToRGB(clamp(v[0], 0, 100), clamp(v[1], 0, 100), clamp(v[2], 0, 100),
clamp(v[3], 0, 100))
    case model == "HLS" && len(v) == 3:
        rgb = hlsToRGB(clamp(v[0], 0, 360), clamp(v[1], 0, 100), clamp(v[2], 0, 100))
    default:
        http.Error(w, "bad model or value count", http.StatusBadRequest)
        return
    }
    for i := range rgb {
        rgb[i] = clamp(rgb[i], 0, 255)
    }
    res := response{
        RGB:  rgb,
        CMYK: rgbToCMYK(rgb[0], rgb[1], rgb[2]),
        HLS:  rgbToHLS(rgb[0], rgb[1], rgb[2]),
```

```
        Hex:  fmt.Sprintf("#%02X%02X%02X", int(math.Round(rgb[0])), int(math.Round(rgb[1])),
int(math.Round(rgb[2]))),
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(res)
}

func main() {
    http.HandleFunc("/api/convert", convert)
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        if r.URL.Path != "/" {
            http.NotFound(w, r)
            return
        }
        w.Header().Set("Content-Type", "text/html; charset=utf-8")
        fmt.Fprint(w, page)
    })
    log.Println("http://localhost:8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}

// Интерфейс: только отображение и ввод; все вычисления делает сервер на Go.
const page = `<!DOCTYPE html>
<html lang="ru"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Цветовые модели</title>
<style>
:root{--bg:#f6f6f9;--card:#fff;--tx:#1d1d28;--mut:#6b6b7b;--bd:#d9d9e3;--ac:#5b4bdb}
@media (prefers-color-scheme:dark){:root{--bg:#15151c;--card:#1f1f29;--tx:#eeeef5;--
mut:#9a9aae;--bd:#34344a;--ac:#8d80ff}}
body{margin:0;background:var(--bg);color:var(--tx);font-family:system-ui,"Segoe UI",Roboto,sans-
serif}
main{max-width:980px;margin:0 auto;padding:16px}
h1{font-size:1.3rem;margin:4px 0 12px}
.top{display:flex;gap:16px;flex-wrap:wrap;align-items:center;background:var(--card);border:1px
solid var(--bd);border-radius:12px;padding:12px}
#sw{width:140px;height:90px;border-radius:10px;border:1px solid var(--bd)}
input[type=color]{width:64px;height:40px;border:none;background:none;padding:0;cursor:pointer}
#hex{font-family:monospace}
.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(280px,1fr));gap:12px;margin-
top:12px}
.m{background:var(--card);border:1px solid var(--bd);border-radius:12px;padding:12px}
.m h2{font-size:1rem;margin:0 0 8px}
.row{display:grid;grid-template-columns:22px 1fr 68px;gap:8px;align-items:center;margin:8px 0}
.row label{font-weight:600;color:var(--mut)}
.row input[type=range]{width:100%;accent-color:var(--ac)}
.row input[type=number]{width:100%;box-sizing:border-box;padding:5px;border:1px solid var(--
bd);border-radius:6px;background:var(--bg);color:var(--tx)}
</style></head><body><main>
<h1>Цветовые модели: CMYK · RGB · HLS</h1>
<div class="top"><div id="sw"></div>
<div><div style="color:var(--mut);font-size:.85rem">Палитра</div><input type="color" id="pal"
value="#5b4bdb"><div id="hex"></div></div></div>
<div class="grid" id="grid"></div>
</main>
<script>
const M={
 CMYK:[["C",100],["M",100],["Y",100],["K",100]],
 RGB:[["R",255],["G",255],["B",255]],
 HLS:[["H",360],["L",100],["S",100]]};
const els={},grid=document.getElementById("grid");
for(const k in M){
 const s=document.createElement("section");s.className="m";s.innerHTML="<h2>"+k+"</
h2>";els[k]=[];
 M[k].forEach(([n,mx])=>{
  const r=document.createElement("div");r.className="row";
```

```
  r.innerHTML="<label>"+n+"</label><input type='range' min='0' max='"+mx+"' step='any'><input
type='number' min='0' max='"+mx+"' step='0.1'>";
  const rg=r.children[1],nm=r.children[2];els[k].push([rg,nm]);
  const h=e=>{let v=parseFloat(e.target.value);if(isNaN(v))return;v=Math.min(mx,Math.max(0,v));
   rg.value=v;if(e.target===rg)nm.value=+v.toFixed(1);send(k)};
  rg.oninput=h;nm.oninput=h;s.appendChild(r)});
 grid.appendChild(s)}
async function send(k,vals){
 vals=vals||els[k].map(e=>parseFloat(e[1].value)||0);
 const res=await fetch("/api/convert?model="+k+"&v="+vals.join(","));
 if(!res.ok)return;const d=await res.json();
 for(const m in M){if(m===k)continue;
  d[m].forEach((x,i)=>{els[m][i][0].value=x;els[m][i][1].value=+x.toFixed(1)})}
 document.getElementById("sw").style.background=d.hex;
 document.getElementById("hex").textContent=d.hex;
 document.getElementById("pal").value=d.hex.toLowerCase()}
document.getElementById("pal").oninput=e=>{const h=e.target.value;
 const rgb=[1,3,5].map(i=>parseInt(h.substr(i,2),16));
 rgb.forEach((x,i)=>{els.RGB[i][0].value=x;els.RGB[i][1].value=x});send("RGB",rgb)};
els.RGB.forEach((e,i)=>{const x=[91,75,219][i];e[0].value=x;e[1].value=x});
send("RGB",[91,75,219]);
</script></body></html>`
```